Actividad 5: Modelado de Datos con Estructuras (struct) y Punteros

Proyecto SISTEMAPROYECTO

Ian Quiroga, Santino Laudati y Dante Iglesias

EEST N.º 1 “Eduardo Ader” - Vicente López

Laboratorio de Programación - 5.º Año 3.º A-B

Profesor: Mansilla Muñoz York Elías

Yamil De Jesus Ganduglia

2026

Fecha de entrega: 4 de noviembre de 2026

Repositorio de GitHub: https://github.com/pachusUwu/sistemaproyecto.git

Introducción

En esta actividad modelamos una entidad del mundo real utilizando una estructura de datos (`struct`) de C++. Elegimos representar un **sensor** de nuestro prototipo para el Proyecto Integrador Anual (P.I.A.), agrupando en una sola entidad su identificador, su nombre o descripción y la lectura que mide.

La carga de los datos se realiza en una función separada que recibe la **dirección de memoria** de la estructura y modifica sus miembros mediante el **operador flecha** (`->`), sin crear copias en la RAM. El informe explica el código, la ejecución, el manejo del buffer de entrada con `cin.ignore()`, el flujo de trabajo con ramas en Git y las respuestas del cuestionario técnico.

Desarrollo

Integrantes y tareas realizadas

Tabla 1

Distribución de tareas del grupo

Integrante	Tareas realizadas
Ian Quiroga	Implementación del `struct EntidadProyecto` y de la función `cargarDatos()`, agregado de la salida del tamaño del struct (`sizeof`), compilación y pruebas.
Santino Laudati	Redacción del informe APA v7, respuestas del cuestionario y del manual del programador (versiones `.md` y `.pdf`).
Dante Iglesias	Repositorio con ramas `main` y `development`, commits descriptivos, README, capturas de ejecución y actualización de la planilla de seguimiento.

Modelo de datos

```
struct EntidadProyecto {
    int   id;          // identificador del sensor
    char  nombre[50];  // descripcion (hasta 49 caracteres + '\0')
    float metrica;     // lectura del sensor (ej. temperatura en grados C)
};
```

La estructura agrupa tres tipos de datos distintos (heterogéneos) bajo una misma entidad lógica, almacenados en un bloque contiguo de memoria. En la ejecución, `sizeof(miEntidad)` informó 60 bytes: 4 del `int`, 50 del arreglo de caracteres, 2 bytes de relleno (*padding*) para alinear el `float` y 4 del `float`.

Pasaje por dirección y operador flecha

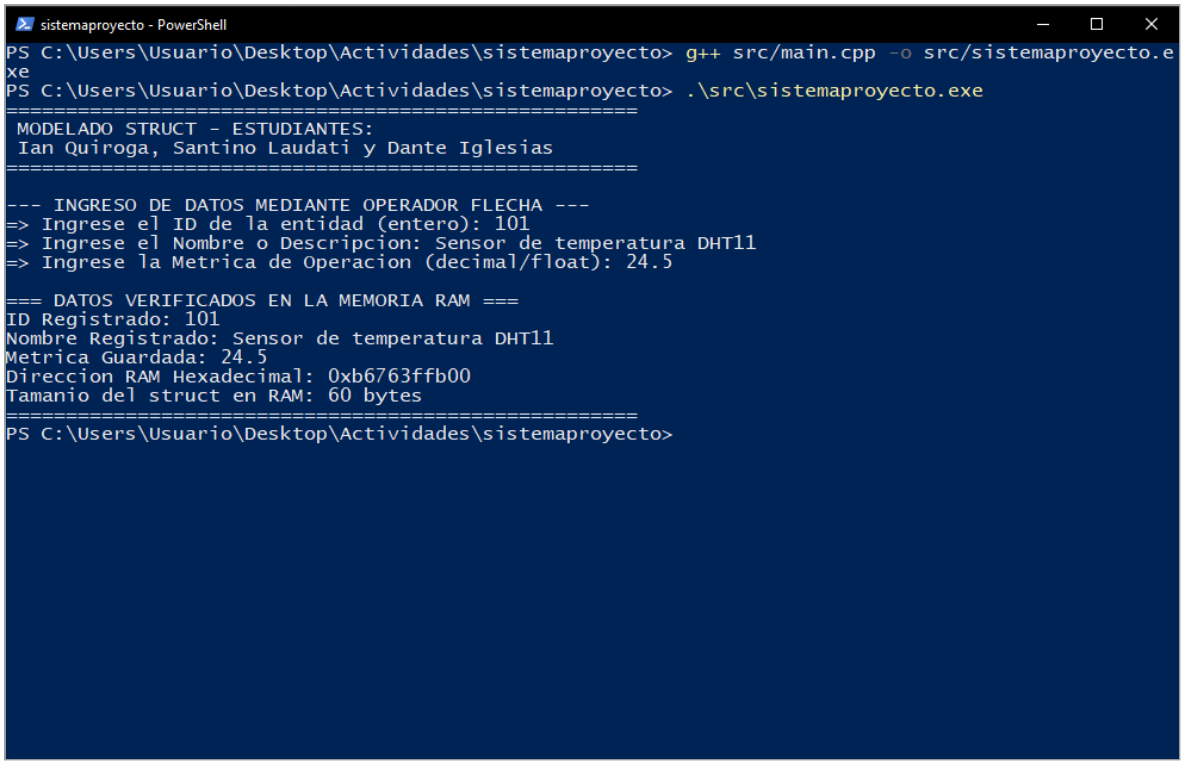
En `main()` se declara `miEntidad` inicializada con valores seguros y se invoca `cargarDatos(&miEntidad);`, enviando la dirección física de la estructura. Dentro de la función, el parámetro `EntidadProyecto* ptr` apunta al original, y cada miembro se completa con el operador flecha:

```
void cargarDatos(EntidadProyecto* ptr) {
    cin >> ptr->id;                 // equivale a (*ptr).id
    cin.ignore();                   // limpia el Enter que quedo en el buffer
    cin.getline(ptr->nombre, 50);   // lee un texto con espacios
    cin >> ptr->metrica;
}
```

Al volver a `main()`, los datos aparecen cargados en `miEntidad` sin haber usado `return`, lo que demuestra que la función modificó la variable original a través del puntero.

Figura 1

Ejecución del modelado con struct: carga del sensor DHT11 mediante el operador flecha



Nota. Se ingresó ID 101, nombre «Sensor de temperatura DHT11» y métrica 24.5. La dirección de la estructura fue 0xb6763ffb00 y su tamaño, 60 bytes.

Comandos utilizados:

```
g++ src/main.cpp -o src/sistemaproyecto.exe
.\src\sistemaproyecto.exe
```

Control de versiones con ramas

El repositorio `sistemaproyecto` se organizó con dos ramas: `main`, que contiene solamente versiones estables, y `development`, donde se fueron registrando los avances con commits descriptivos (estructura del proyecto, código del struct, documentación y capturas). Al terminar, `development` se integró a `main` mediante un *merge*, dejando un historial ramificado visible con `git log --graph`.

Cuestionario técnico de control

1. Heterogeneidad. Un `struct` permite agrupar datos de distinto tipo que pertenecen a una misma cosa. En lugar de tener variables sueltas como `idSensor1`, `nombreSensor1` y `lecturaSensor1`, se tiene una sola variable `sensor` con todos sus datos

juntos. Esto hace el código más ordenado, evita mezclar datos de dos sensores distintos, permite pasar todo el dispositivo a una función con un único parámetro y facilita crear arreglos de sensores (`EntidadProyecto sensores[10];`).

2. Operador flecha. El operador punto (`.`) se usa con una variable de tipo estructura, pero `ptr` no es una estructura: es una dirección de memoria. Primero hay que desreferenciar el puntero para llegar a la estructura y recién ahí acceder al miembro, es decir `(*ptr).id`. El operador flecha `ptr->id` es la forma abreviada de esa misma operación. Escribir `ptr.id` es un error de compilación porque un puntero no tiene miembros.

3. Eficiencia en RAM. En el pasaje por copia, la función recibe un duplicado completo de la estructura (en nuestro caso 60 bytes, y podrían ser muchos más), lo que consume memoria y tiempo para copiarla; además, los cambios se hacen sobre la copia y se pierden al terminar la función. Con el pasaje por referencia mediante puntero solo se envía la dirección (8 bytes en un sistema de 64 bits), sin importar el tamaño de la estructura, y la función trabaja directamente sobre el original.

4. Buffer de entrada. Después de `cin >> ptr->id;` el número se lee, pero el Enter (`\n`) que presionó el usuario queda guardado en el buffer. Si se omite `cin.ignore()`, el `cin.getline()` siguiente encuentra ese `\n` inmediatamente, lo interpreta como una línea vacía y termina sin esperar a que el usuario escriba: en la terminal se ve que el programa "saltea" la pregunta del nombre y el campo queda vacío.

5. Aplicación al Proyecto Integrador. Representamos un sensor de temperatura y humedad (DHT11) del prototipo del P.I.A. Sus miembros son `id` (número de sensor dentro del sistema), `nombre` (descripción y ubicación, por ejemplo "Sensor de temperatura DHT11") y `metrica` (la última lectura en grados Celsius). Este mismo modelo puede ampliarse con campos como unidad de medida, umbral de alarma o fecha y hora de la lectura.

Conclusión

La actividad nos mostró cómo pasar de variables sueltas a un modelo de datos organizado. El `struct` nos permitió representar un sensor completo como una sola entidad, y el pasaje por dirección con el operador flecha nos permitió modificarlo desde una función sin copias innecesarias.

Aprendimos además un detalle práctico muy común: el Enter que queda en el buffer después de leer un número, que se resuelve con `cin.ignore()`. Por último, trabajar con las ramas `main` y `development` nos acercó a la forma en que se organizan los equipos de desarrollo reales, y esta estructura servirá como base para modelar los datos de nuestro Proyecto Integrador.

Referencias

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

cplusplus.com. (s.f.). *Data structures*. https://cplusplus.com/doc/tutorial/structures/

cplusplus.com. (s.f.). *istream::ignore*. https://cplusplus.com/reference/istream/istream/ignore/

Mansilla Muñoz, Y. E. (2026). *Actividad 5: Modelado de datos con estructuras (struct) y punteros* [Material de clase]. E.E.S.T. N.° 1 "Eduardo Ader".

Quiroga, I., Laudati, S., & Iglesias, D. (2026). *sistemaproyecto* [Repositorio de código fuente]. GitHub. https://github.com/pachusUwu/sistemaproyecto